

HW9: Individual Effort Report

Server Design

I. Overview of Major Components and Packages

1. Server.java (Servlet)

- Handles HTTP requests (GET/POST).
- Interacts with the MySQL database.
- Publishes review events to RabbitMQ for asynchronous processing.

2. AlbumInfo.java

- Model class representing album metadata.
- Used as a data transfer object (DTO) for serializing/deserializing album data (via Gson).

3. ReviewConsumer.java

- Consumes messages from RabbitMQ (likeQueue).
- Parses and stores user reviews into the MySQL album_reviews table.

II. Data Flow and Relationships

1. Album Upload

- Stores the album into the albums table in MySQL.
- Returns AlbumInfo as a JSON response.

2. Review Upload

- Publishes it to RabbitMQ queue likeQueue.

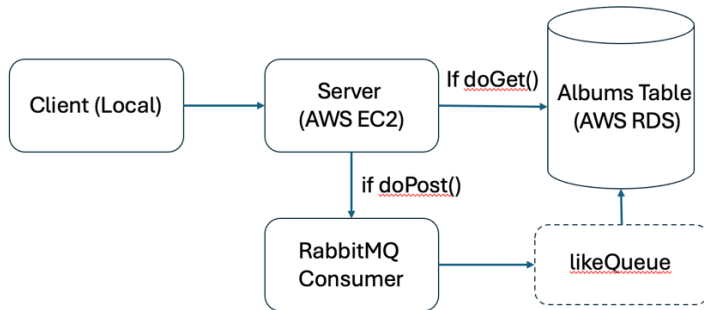
3. Review Consumption (RabbitMQ Consumer)

- On message receive, inserts or updates the review in the MySQL album_reviews table.

4. Review Get

- Get random album review stats in JSON

III. Server on AWS



(Note: RabbitMQ runs locally via an SSH channel directing messages)

Results (locally)

The throughput has been steadily kept at around 2000 in each experiment, except it jumped to around 3000 in 10 10 2, meaning that the system may have some room to grow, it could be due to queued tasks can be fed more consistently or more parallelism fills in idle CPU/network gaps, or simple the network behaves better at that time. The sure thing right now is that the system may still have room for handling more threads.

```
DISLINE SUCCESS FOR ALBUM ID: 427786
Created Album ID: 427786
LIKE Success for Album ID: 427786
LIKE Success for Album ID: 427786
DISLINE Success for Album ID: 427786
SLF4J: Failed to load class "org.slf4j.impl.StaticMDCBinder".
SLF4J: Defaulting to no-operation MDCAdapter implementation.
SLF4J: See http://www.slf4j.org/codes.html#no_static_mdc_binder for further details.
[RUN CONFIG]: 10 10 2
GET Mean Latency: 0.374440351374327
GET Min Latency: 0
GET Max Latency: 4
GET 50th Percentile: 0.0
GET 99th Percentile: 1.0
POST Mean Latency: 4.4451
POST Min Latency: 1
POST Max Latency: 232
POST 50th Percentile: 4.0
POST 99th Percentile: 18.0
Time taken: 25s
Throughput: 3011 requests/s
Throughput: 1680 HTTP POST requests/s
Throughput: 7055.177928828469 GET requests/s
Number of successful GET requests: 35290
Number of failed GET requests: 0
Number of successful POST requests: 40800
Number of failed POST requests: 0
Process finished with exit code 0

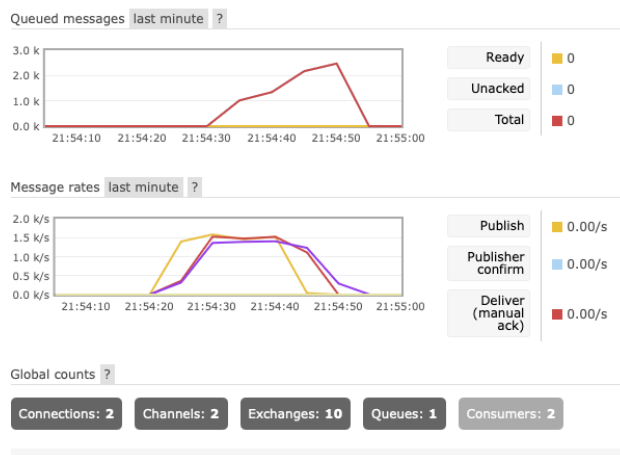
SLF4J: Defaulting to no-operation MDCAdapter implementation.
SLF4J: See http://www.slf4j.org/codes.html#no_static_mdc_binder for further details.
[RUN CONFIG]: 10 20 2
GET Mean Latency: 0.6293835184556434
GET Min Latency: 0
GET Max Latency: 9
GET 50th Percentile: 1.0
GET 99th Percentile: 2.0
POST Mean Latency: 4.702525
POST Min Latency: 1
POST Max Latency: 93
POST 50th Percentile: 4.0
POST 99th Percentile: 16.0
Time taken: 44s
Throughput: 2286 requests/s
Throughput: 1818 HTTP POST requests/s
Throughput: 4119.2807192807195 GET requests/s
Number of successful GET requests: 20617
Number of failed GET requests: 0
Number of successful POST requests: 80800
Number of failed POST requests: 0
Process finished with exit code 0
```

```
Run Main x
SLF4J: Defaulting to no-operation MDCAdapter implementation.
SLF4J: See http://www.slf4j.org/codes.html#no-static_mdc_binder for
[RUN CONFIG]: 10 30 2
GET Mean Latency: 0.9421168506042829
GET Min Latency: 0
GET Max Latency: 10
GET 50th Percentile: 1.0
GET 99th Percentile: 3.0
POST Mean Latency: 4.583325
POST Min Latency: 3
POST Max Latency: 187
POST 50th Percentile: 4.0
POST 99th Percentile: 14.0
Time taken: 64s
Throughput: 2096 requests/s
Throughput: 1875 HTTP POST requests/s
Throughput: 2826.4083100279663 GET requests/s
Number of successful GET requests: 14149
Number of failed GET requests: 0
Number of successful POST requests: 120000
Number of failed POST requests: 0
Process finished with exit code 0
client > src > main > java > Main > DELAY
```

However, RabbitMQ queue sizes increased significantly. Before implementing the 3 getReview threads, the max size was 75. Right now the queue size easily reached 2000, and max at around 7500 in 10302.

Possible reasons maybe that before adding the threads, we had only POST threads creating albums and immediately queuing album IDs so that each post thread finished fast and the album ID queue (albumIds) grew slowly and stopped growing once POST threads finished. After adding the threads, all albums created during POST are already queued by then. And the get review threads Continuously read album IDs *without removing them* from the queue and do not consume(resume) IDs, and allow the queue to grow indefinitely.

RabbitMQ statistics for 10 10 2:

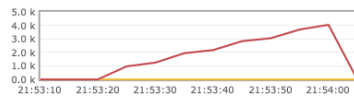


RabbitMQ statistics for 10 20 2:

Overview

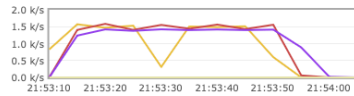
Totals

Queued messages **last minute** ?



Ready 0
Unacked 0
Total 0

Message rates **last minute** ?



Publish 0.00/s
Publisher confirm 0.00/s
Deliver (manual ack) 0.00/s

Global counts ?

Connections: 2 Channels: 2 Exchanges: 10 Queues: 1 Consumers: 2

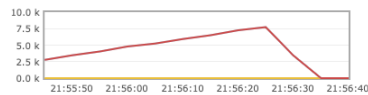
Nodes

RabbitMQ statistics for 10 30 2:

Overview

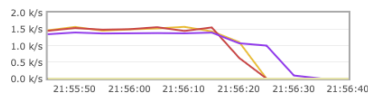
Totals

Queued messages **last minute** ?



Ready 0
Unacked 0
Total 0

Message rates **last minute** ?



Publish 0.00/s
Publisher confirm 0.00/s
Deliver (manual ack) 0.00/s

Global counts ?

Connections: 2 Channels: 2 Exchanges: 10 Queues: 1 Consumers: 2

Nodes

Results (AWS EC2)

The throughput has been steadily kept at around 120 in each experiment, except the connection between local client and server on AWS EC2 stopped during running 10302, resulting in unknown throughput. The sure thing is that the throughput of 10302 is highly unlikely to be better due to queue size. The reason of the dramatically declined throughput may due to the capacity of the AWS EC2 instance, which is of type t2.micro, often good for low-load, burstable tasks. Specifically, the type of GPU and RAM size would have great impact on the program. In this program, Spark, HTTP client threads, Tomcat + Servlet, MySQL connections, and RabbitMQ need to use RAM. Although not all of them used the EC2 RAM, the majority uses and will result in system crash easily, which happened.

```
SLF4J: Failed to load class "org.slf4j.impl.StaticMDCBinder".
SLF4J: Defaulting to no-operation MDCAdapter implementation.
SLF4J: See http://www.slf4j.org/codes.html#no\_static\_mdc\_binder for fu

[ RUN CONFIG]: 10 10 2
GET Mean Latency: 20.627615062761507
GET Min Latency: 14
GET Max Latency: 278
GET 50th Percentile: 18.0
GET 99th Percentile: 48.0
POST Mean Latency: 434.01474466855893
POST Min Latency: 22
POST Max Latency: 32860
POST 50th Percentile: 316.0
POST 99th Percentile: 3493.0
Time taken: 336s
Throughput: 121 requests/s
Throughput: 119 HTTP POST requests/s
Throughput: 142.51639833035182 GET requests/s
Number of successful GET requests: 717
Number of failed GET requests: 0
Number of successful POST requests: 39811
Number of failed POST requests: 183
Process finished with exit code 0

client > 10_10_2.csv

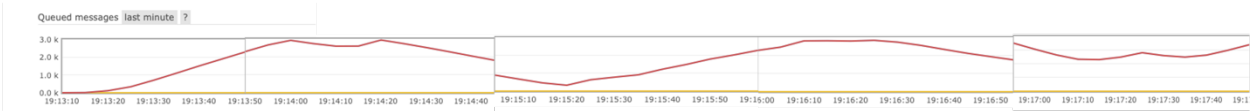
SLF4J: Failed to load class "org.slf4j.impl.StaticMDCBinder".
SLF4J: Defaulting to no-operation MDCAdapter implementation.
SLF4J: See http://www.slf4j.org/codes.html#no\_static\_mdc\_binder for fu

[ RUN CONFIG]: 10 20 2
GET Mean Latency: 6963.2
GET Min Latency: 24
GET Max Latency: 23017
GET 50th Percentile: 101.0
GET 99th Percentile: 23817.0
POST Mean Latency: 773.0681512981199
POST Min Latency: 23
POST Max Latency: 36189
POST 50th Percentile: 365.0
POST 99th Percentile: 15779.0
Time taken: 621s
Throughput: 119 requests/s
Throughput: 119 HTTP POST requests/s
Throughput: 0.4304963623057385 GET requests/s
Number of successful GET requests: 10
Number of failed GET requests: 0
Number of successful POST requests: 71488
Number of failed POST requests: 2638
Process finished with exit code 0

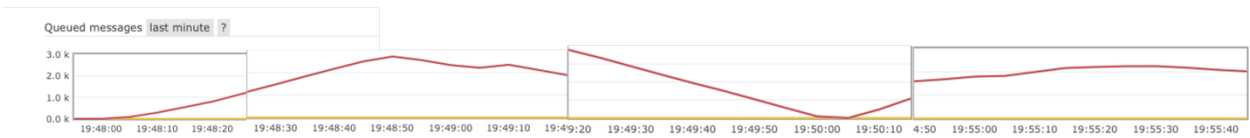
client > src > main > java > Main > main
```

Because we had a lower performance machine, we can see that the queue size also increased. It easily reached around 5000, and the min process rate can reach 1.6 requests/s while the total number of successful get requests is 10. There could be a couple of reasons: 1. There is additional overhead on SSH tunneling because I am running RabbitMQ locally and the server need to direct message via an SSH tunnel. The SSH tunnels add encryption, and result in higher latency. 2. The RabbitMQ is single-threaded. This could be the biggest contributor to the result as the queue size did not vary as significantly as the throughput comparing with performance on running the program locally and on VM.

RabbitMQ statistics for 10 10 2:



RabbitMQ statistics for 10 20 2:



RabbitMQ statistics for 10 30 2:

